

EJERCICIOS RESUELTOS

Arrays · POO · Excepciones

FICHA — EJERCICIOS ARRAYS

Ejercicio 1 — Crear y mostrar un array

Crea un array de 5 enteros, asigna valores y muéstralos con for tradicional y foreach.

```
public class Ej1 {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30, 40, 50};

        // a) for tradicional
        System.out.println("--- For tradicional ---");
        for (int i = 0; i < numeros.length; i++) {
            System.out.println("Posición " + i + ": " + numeros[i]);
        }

        // b) foreach
        System.out.println("--- Foreach ---");
        for (int n : numeros) {
            System.out.println(n);
        }
    }
}
```

Ejercicio 2 — Suma y media

Crea un array de 6 reales. Calcula la suma total y la media.

```
public class Ej2 {
    public static void main(String[] args) {
        double[] valores = {3.5, 7.0, 2.5, 8.0, 5.5, 4.0};
        double suma = 0;

        for (double v : valores) {
            suma += v;
        }

        double media = suma / valores.length;
        System.out.println("Suma: " + suma);
        System.out.println("Media: " + media);
    }
}
```

Ejercicio 3 — Contar números positivos

Dado int[] numeros = {4, -3, 7, 0, -2, 8}; cuenta cuántos son positivos.

```
public class Ej3 {
    public static void main(String[] args) {
        int[] numeros = {4, -3, 7, 0, -2, 8};
        int positivos = 0;

        for (int n : numeros) {
            if (n > 0) positivos++;
        }

        System.out.println("Positivos: " + positivos); // 3
    }
}
```

Ejercicio 4 — Buscar un número

Pide un número por teclado e indica si está en el array y en qué posición.

```
import java.util.Scanner;

public class Ej4 {
    public static void main(String[] args) {
        int[] numeros = {5, 3, 8, 1, 9, 2};
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce un número a buscar: ");
        int buscar = sc.nextInt();
        boolean encontrado = false;

        for (int i = 0; i < numeros.length; i++) {
            if (numeros[i] == buscar) {
                System.out.println("Encontrado en posición: " + i);
                encontrado = true;
                break;
            }
        }

        if (!encontrado) System.out.println("No está en el array");
    }
}
```

Ejercicio 5 — Encontrar el mayor

Crea un array de 10 enteros y muestra el número mayor.

```
public class Ej5 {
    public static void main(String[] args) {
        int[] numeros = {4, 7, 2, 9, 1, 6, 3, 8, 5, 0};
        int mayor = numeros[0];

        for (int i = 1; i < numeros.length; i++) {
            if (numeros[i] > mayor) mayor = numeros[i];
        }

        System.out.println("El mayor es: " + mayor); // 9
    }
}
```

Ejercicio 6 — Contar pares e impares

Dado un array de enteros, cuenta cuántos son pares y cuántos impares.

```
public class Ej6 {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5, 6, 7, 8};
        int pares = 0, impares = 0;

        for (int n : numeros) {
            if (n % 2 == 0) pares++;
            else impares++;
        }

        System.out.println("Pares: " + pares);
        System.out.println("Impares: " + impares);
    }
}
```

Ejercicio 7 — Invertir el array

Dado {1,2,3,4,5}, crea otro array con el orden invertido.

```
public class Ej7 {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5};
        int[] invertido = new int[numeros.length];

        for (int i = 0; i < numeros.length; i++) {
            invertido[i] = numeros[numeros.length - 1 - i];
        }

        for (int n : invertido) {
            System.out.print(n + " ");
        }
        // Salida: 5 4 3 2 1
    }
}
```

Ejercicio 8 — Copiar un array manualmente

Sin usar clone() ni Arrays.copyOf(), copia un array en otro con un for.

```
public class Ej8 {
    public static void main(String[] args) {
        int[] origen = {10, 20, 30, 40, 50};
        int[] destino = new int[origen.length];

        for (int i = 0; i < origen.length; i++) {
            destino[i] = origen[i];
        }

        // Verificar que son independientes
        origen[0] = 999;
        System.out.println(origen[0]); // 999
        System.out.println(destino[0]); // 10 (no cambia)
    }
}
```

Ejercicio 9 — Comparar arrays: equals vs Arrays.equals

Comprueba si dos arrays son iguales con equals y con Arrays.equals. Explica la diferencia.

```
import java.util.Arrays;

public class Ej9 {
    public static void main(String[] args) {
        int[] a = {1, 2, 3};
        int[] b = {1, 2, 3};

        // equals compara REFERENCIAS (si son el mismo objeto en memoria)
        System.out.println(a.equals(b)); // false

        // Arrays.equals compara el CONTENIDO elemento a elemento
        System.out.println(Arrays.equals(a, b)); // true

        // DIFERENCIA: equals() en arrays hereda de Object y solo
```

```

        // devuelve true si a y b apuntan al mismo objeto.
        // Arrays.equals() compara cada posición del array.
    }
}

```

Ejercicio 10 — Probar clone()

Crea un array, clónalo y modifica el original. Muestra ambos y explica qué ocurre.

```

import java.util.Arrays;

public class Ej10 {
    public static void main(String[] args) {
        int[] original = {1, 2, 3, 4, 5};
        int[] clon     = original.clone();

        original[0] = 99;

        System.out.println(Arrays.toString(original)); // [99, 2, 3, 4, 5]
        System.out.println(Arrays.toString(clon));      // [1, 2, 3, 4, 5]

        // clone() hace una copia independiente (copia superficial).
        // Modificar el original NO afecta al clon.
    }
}

```

Ejercicio 11 — Método que SÍ modifica el array

Crea el método `multiplicarPorDos(int[] array)` y comprueba que el array original cambia.

```

import java.util.Arrays;

public class Ej11 {
    public static void multiplicarPorDos(int[] array) {
        for (int i = 0; i < array.length; i++) {
            array[i] *= 2;
        }
    }

    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5};
        multiplicarPorDos(numeros);
        System.out.println(Arrays.toString(numeros)); // [2, 4, 6, 8, 10]
        // El array original SÍ cambia porque se pasa la referencia.
    }
}

```

Ejercicio 12 — Método que NO modifica el array

Crea un método que use `foreach` y multiplique por 2 cada elemento. ¿Cambia el array?

```

import java.util.Arrays;

public class Ej12 {
    public static void intentarMultiplicar(int[] array) {
        for (int n : array) {
            n *= 2; // n es una COPIA local del valor, no la posición del array
        }
    }
}

```

```

public static void main(String[] args) {
    int[] numeros = {1, 2, 3, 4, 5};
    intentarMultiplicar(numeros);
    System.out.println(Arrays.toString(numeros)); // [1, 2, 3, 4, 5]
    // El array NO cambia. El foreach copia el valor en la variable n,
    // modificar n no modifica la posición original del array.
}
}

```

Ejercicio 13 — Array de objetos Alumno

Crea la clase Alumno con nombre y nota. Crea un array de 3 alumnos y muestra sus datos.

```

public class Alumno {
    private String nombre;
    private double nota;

    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    public String getNombre() { return nombre; }
    public double getNota() { return nota; }
    public void setNota(double nota) { this.nota = nota; }

    @Override
    public String toString() {
        return nombre + " -> " + nota;
    }
}

public class Ej13 {
    public static void main(String[] args) {
        Alumno[] alumnos = new Alumno[3];
        alumnos[0] = new Alumno("Ana", 8.5);
        alumnos[1] = new Alumno("Luis", 6.0);
        alumnos[2] = new Alumno("Marta", 9.0);

        for (Alumno a : alumnos) {
            System.out.println(a);
        }
    }
}

```

Ejercicio 14 — Alumno con mejor nota

Dado un array de Alumno, muestra el alumno con mayor nota usando compareTo.

```

public class Ej14 {
    public static void main(String[] args) {
        Alumno[] alumnos = {
            new Alumno("Ana", 8.5),
            new Alumno("Luis", 6.0),
            new Alumno("Marta", 9.0)
        };

        Alumno mejor = alumnos[0];

        for (int i = 1; i < alumnos.length; i++) {
            if (alumnos[i].compareTo(mejor) > 0) {
                mejor = alumnos[i];
            }
        }
    }
}

```

```

        }
    }

    System.out.println("Mejor alumno: " + mejor);
}

// Clase Alumno debe implementar Comparable:
public class Alumno implements Comparable<Alumno> {
    // ... (mismos atributos y constructor del Ej13)

    @Override
    public int compareTo(Alumno otro) {
        if (this.nota > otro.nota)    return 1;
        else if (this.nota < otro.nota) return -1;
        else                          return 0;
    }
}

```

Ejercicio 15 — Subir nota a todos

Recorre el array de alumnos y suma 1 punto a todos.

```

public class Ej15 {
    public static void main(String[] args) {
        Alumno[] alumnos = {
            new Alumno("Ana", 8.5),
            new Alumno("Luis", 6.0),
            new Alumno("Marta", 9.0)
        };

        for (Alumno a : alumnos) {
            a.setNota(a.getNota() + 1);
        }

        for (Alumno a : alumnos) {
            System.out.println(a);
        }
    }
}

```

Ejercicio 16 — Ordenar alumnos por nota con Arrays.sort

Haz que Alumno implemente Comparable<Alumno> y ordena el array por nota con Arrays.sort.

```

import java.util.Arrays;

public class Alumno implements Comparable<Alumno> {
    private String nombre;
    private double nota;

    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    public String getNombre() { return nombre; }
    public double getNota() { return nota; }
    public void setNota(double nota) { this.nota = nota; }

    @Override
    public int compareTo(Alumno otro) {
        if (this.nota > otro.nota)    return 1;
    }
}

```

```

        else if (this.nota < otro.nota) return -1;
        else return 0;
    }

    @Override
    public String toString() { return nombre + " -> " + nota; }
}

public class Ej16 {
    public static void main(String[] args) {
        Alumno[] alumnos = {
            new Alumno("Ana", 8.5),
            new Alumno("Luis", 6.0),
            new Alumno("Marta", 9.0),
            new Alumno("Pedro", 7.0)
        };

        Arrays.sort(alumnos); // usa compareTo internamente

        for (Alumno a : alumnos) {
            System.out.println(a);
        }
        // Salida (ascendente): Luis->6.0, Pedro->7.0, Ana->8.5, Marta->9.0
    }
}

```


RELACIÓN 7 — POO II con Excepciones

Ejercicio 1 — Clase MiEntradaSalida

Clase con métodos estáticos para leer entero, entero positivo, entero en rango, carácter, S/N y cadena, repitiendo si hay error.

```
import java.util.Scanner;

public class MiEntradaSalida {
    private static Scanner sc = new Scanner(System.in);

    public static int solicitarEntero(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje);
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Error: introduce un número entero.");
            }
        }
    }

    public static int solicitarEnteroPositivo(String mensaje) {
        while (true) {
            int n = solicitarEntero(mensaje);
            if (n > 0) return n;
            System.out.println("Error: el número debe ser positivo.");
        }
    }

    public static int solicitarEnteroEnRango(String mensaje, int min, int max) {
        while (true) {
            int n = solicitarEntero(mensaje);
            if (n >= min && n <= max) return n;
            System.out.println("Error: debe estar entre " + min + " y " + max);
        }
    }

    public static char solicitarCaracter(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje);
                String linea = sc.nextLine().trim();
                if (linea.isEmpty()) throw new Exception();
                return linea.charAt(0);
            } catch (Exception e) {
                System.out.println("Error: introduce un carácter.");
            }
        }
    }

    public static char solicitarCaracterSN(String mensaje) {
        while (true) {
            char c = Character.toUpperCase(solicitarCaracter(mensaje));
            if (c == 'S' || c == 'N') return c;
            System.out.println("Error: introduce S o N.");
        }
    }

    public static String solicitarCadena(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje);
                String s = sc.nextLine().trim();
                if (!s.isEmpty()) return s;
                System.out.println("Error: la cadena no puede estar vacía.");
            } catch (Exception e) {
                System.out.println("Error al leer.");
            }
        }
    }
}
```

```

    }
}

// Clase Principal para probar:
public class Principal {
    public static void main(String[] args) {
        int n = MiEntradaSalida.solicitarEntero("Entero: ");
        int np = MiEntradaSalida.solicitarEnteroPositivo("Positivo: ");
        int nr = MiEntradaSalida.solicitarEnteroEnRango("Rango 1-10: ", 1, 10);
        char c = MiEntradaSalida.solicitarCaracter("Carácter: ");
        char sn = MiEntradaSalida.solicitarCaracterSN("S/N: ");
        String s = MiEntradaSalida.solicitarCadena("Cadena: ");

        System.out.println(n + " " + np + " " + nr + " " + c + " " + sn + " " + s);
    }
}

```

Ejercicio 2 — Clase Punto y Línea con menú

Clase Punto(x,y) y clase Línea(puntoA, puntoB) con métodos de movimiento y menú interactivo.

```

public class Punto {
    private double x;
    private double y;

    public Punto(double x, double y) { this.x = x; this.y = y; }

    public double getX() { return x; }
    public double getY() { return y; }
    public void setX(double x) { this.x = x; }
    public void setY(double y) { this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Punto)) return false;
        Punto otro = (Punto) obj;
        return this.x == otro.x && this.y == otro.y;
    }

    @Override
    public String toString() { return "(" + x + "," + y + ")"; }
}

public class Línea {
    private Punto puntoA;
    private Punto puntoB;

    public Línea(Punto a, Punto b) {
        if (a.equals(b))
            throw new IllegalArgumentException("Los dos puntos no pueden ser iguales");
        this.puntoA = a;
        this.puntoB = b;
    }

    public Punto getPuntoA() { return puntoA; }
    public Punto getPuntoB() { return puntoB; }

    public void moverDerecha(double d) { puntoA.setX(puntoA.getX()+d); puntoB.setX(puntoB.getX()+d); }
    public void moverIzquierda(double d) { puntoA.setX(puntoA.getX()-d); puntoB.setX(puntoB.getX()-d); }
    public void moverArriba(double d) { puntoA.setY(puntoA.getY()+d); puntoB.setY(puntoB.getY()+d); }
}

```

```

    public void moverAbajo(double d) { puntoA.setY(puntoA.getY() -d);
    puntoB.setY(puntoB.getY() -d); }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Linea)) return false;
        Linea otra = (Linea) obj;
        return puntoA.equals(otra.puntoA) && puntoB.equals(otra.puntoB);
    }

    @Override
    public String toString() { return "[" + puntoA + "," + puntoB + "];" }
}

// Main con menú:
import java.util.Scanner;
public class MainLinea {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Punto A - x: "); double ax =
        Double.parseDouble(sc.nextLine());
        System.out.print("Punto A - y: "); double ay =
        Double.parseDouble(sc.nextLine());
        System.out.print("Punto B - x: "); double bx =
        Double.parseDouble(sc.nextLine());
        System.out.print("Punto B - y: "); double by =
        Double.parseDouble(sc.nextLine());

        Linea linea = new Linea(new Punto(ax, ay), new Punto(bx, by));
        int opcion;

        do {
            System.out.println("1.Mover 2.Mostrar 3.Salir");
            opcion = Integer.parseInt(sc.nextLine());
            switch (opcion) {
                case 1:
                    System.out.print("Dirección (A/B/I/D): ");
                    char dir = sc.nextLine().toUpperCase().charAt(0);
                    System.out.print("Distancia: ");
                    double dist = Double.parseDouble(sc.nextLine());
                    if (dir=='A') linea.moverArriba(dist);
                    else if (dir=='B') linea.moverAbajo(dist);
                    else if (dir=='I') linea.moverIzquierda(dist);
                    else if (dir=='D') linea.moverDerecha(dist);
                    break;
                case 2: System.out.println(linea); break;
                case 3: System.out.println("Saliendo..."); break;
                default: System.out.println("Opción inválida");
            }
        } while (opcion != 3);
    }
}

```

Ejercicio 3 — Producto con Categoría

Clase Categoría con nombre e IVA. Clase Producto con código automático, descripción, precio sin IVA y categoría. Método para calcular precio con IVA.

```

public class Categoria {
    private String nombre;
    private double iva; // porcentaje, ej: 21.0

    public Categoria(String nombre, double iva) {
        if (iva < 0) throw new IllegalArgumentException("IVA no puede ser negativo");
        this.nombre = nombre;
        this.iva = iva;
    }
}

```

```

    public String getNombre() { return nombre; }
    public double getIva() { return iva; }

    @Override
    public String toString() { return nombre + " (IVA " + iva + "%)"; }
}

public class Producto {
    private static int contadorCodigo = 0;
    private int codigo;
    private String descripcion;
    private double precioSinIva;
    private Categoria categoria;

    public Producto(String descripcion, double precioSinIva, Categoria categoria) {
        if (precioSinIva < 0) throw new IllegalArgumentException("Precio no puede ser negativo");
        contadorCodigo++;
        this.codigo = contadorCodigo;
        this.descripcion = descripcion;
        this.precioSinIva = precioSinIva;
        this.categoria = categoria;
    }

    public double getPrecioConIva() {
        return precioSinIva * (1 + categoria.getIva() / 100);
    }

    public int getCodigo() { return codigo; }
    public String getDescripcion() { return descripcion; }
    public double getPrecioSinIva() { return precioSinIva; }
    public Categoria getCategoria() { return categoria; }

    public void setPrecioSinIva(double p) {
        if (p >= 0) this.precioSinIva = p;
    }

    @Override
    public String toString() {
        return "[" + codigo + "] " + descripcion
            + " | Sin IVA: " + precioSinIva
            + "€ | Con IVA: " + String.format("%.2f", getPrecioConIva()) + "€";
    }
}

public class MainProducto {
    public static void main(String[] args) {
        Categoria alimentacion = new Categoria("Alimentación", 4.0);
        Categoria electronica = new Categoria("Electrónica", 21.0);

        Producto p1 = new Producto("Pan", 1.00, alimentacion);
        Producto p2 = new Producto("Portátil", 800.0, electronica);
        Producto p3 = new Producto("Leche", 0.90, alimentacion);

        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);
    }
}

```

Ejercicio 4 — Clase Jarra con menú

Simulación de jarras: llenar, vaciar, volcar. Control del total de agua gastada (static).

```

public class Jarra {
    private static double totalAguaGastada = 0;

```

```

    private double capacidad;
    private double cantidad;

    public Jarra(double capacidad) {
        if (capacidad <= 0) throw new IllegalArgumentException("Capacidad debe ser
positiva");
        this.capacidad = capacidad;
        this.cantidad = 0;
    }

    public void llenar() {
        double aguaAnadir = capacidad - cantidad;
        cantidad = capacidad;
        totalAguaGastada += aguaAnadir;
    }

    public void vaciar() {
        cantidad = 0;
    }

    public void volcarEn(Jarra otra) {
        double espacio = otra.capacidad - otra.cantidad;
        if (this.cantidad <= espacio) {
            otra.cantidad += this.cantidad;
            this.cantidad = 0;
        } else {
            this.cantidad -= espacio;
            otra.cantidad = otra.capacidad;
        }
    }

    public static double getTotalAguaGastada() { return totalAguaGastada; }

    @Override
    public String toString() {
        return "Capacidad: " + capacidad + "L | Contenido: " + cantidad + "L";
    }
}

import java.util.Scanner;
public class MainJarra {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Capacidad jarra A: "); double ca =
Double.parseDouble(sc.nextLine());
        System.out.print("Capacidad jarra B: "); double cb =
Double.parseDouble(sc.nextLine());
        Jarra A = new Jarra(ca);
        Jarra B = new Jarra(cb);
        int opcion;

        do {
            System.out.println("1.Llenar  2.Vaciar  3.A->B  4.B->A  5.Estado
6.Salir");
            opcion = Integer.parseInt(sc.nextLine());
            switch (opcion) {
                case 1:
                    System.out.print("¿Qué jarra llenar (A/B)? ");
                    char j1 = sc.nextLine().toUpperCase().charAt(0);
                    if (j1 == 'A') A.llenar(); else B.llenar();
                    break;
                case 2:
                    System.out.print("¿Qué jarra vaciar (A/B)? ");
                    char j2 = sc.nextLine().toUpperCase().charAt(0);
                    if (j2 == 'A') A.vaciar(); else B.vaciar();
                    break;
                case 3: A.volcarEn(B); break;
                case 4: B.volcarEn(A); break;
                case 5:
                    System.out.println("A: " + A);
                    System.out.println("B: " + B);

```

```

        break;
        case 6: break;
        default: System.out.println("Opción inválida");
    }
} while (opcion != 6);

System.out.println("Total agua gastada: " + Jarra.getTotalAguaGastada() + "
litros");
}
}

```

Ejercicio 5 — Liga de fútbol

Clases Equipo y Partido. Método ponerResultado con manejo de excepciones.

```

public class Equipo {
    private String nombre;
    private String estadio;
    private String ciudad;
    private int    partidosGanados;

    public Equipo(String nombre, String estadio, String ciudad) {
        this.nombre     = nombre;
        this.estadio     = estadio;
        this.ciudad      = ciudad;
        this.partidosGanados = 0;
    }

    public String getNombre() { return nombre; }
    public String getEstadio() { return estadio; }
    public String getCiudad() { return ciudad; }
    public int    getGanados() { return partidosGanados; }
    public void   sumarGanado(){ partidosGanados++; }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Equipo)) return false;
        return this.nombre.equals(((Equipo) obj).nombre);
    }

    @Override
    public String toString() {
        return nombre + " | Estadio: " + estadio + " | Ganados: " + partidosGanados;
    }
}

public class Partido {
    private final int    jornada;
    private final Equipo local;
    private final Equipo visitante;
    private int    golesLocal    = -1;
    private int    golesVisitante = -1;
    private char   quiniela      = ' ';
    private boolean jugado       = false;

    public Partido(int jornada, Equipo local, Equipo visitante) {
        if (jornada < 1 || jornada > 38)
            throw new IllegalArgumentException("Jornada debe estar entre 1 y 38");
        if (local.equals(visitante))
            throw new IllegalArgumentException("Los equipos deben ser distintos");
        this.jornada = jornada;
        this.local    = local;
        this.visitante = visitante;
    }

    public void ponerResultado(String resultado) throws Exception {
        try {

```

```

        String[] partes = resultado.split("-");
        if (partes.length != 2) throw new Exception("Formato inválido. Usa 'X-Y'");

        int gl = Integer.parseInt(partes[0].trim());
        int gv = Integer.parseInt(partes[1].trim());
        if (gl < 0 || gv < 0) throw new Exception("Los goles no pueden ser negativos");

        this.golesLocal      = gl;
        this.golesVisitante = gv;
        this.jugado          = true;

        if (gl > gv) { quiniela = '1'; local.sumarGanado(); }
        else if (gv > gl) { quiniela = '2'; visitante.sumarGanado(); }
        else { quiniela = 'X'; }

    } catch (NumberFormatException e) {
        throw new Exception("Los goles deben ser números enteros");
    }
}

@Override
public String toString() {
    if (!jugado) {
        return "Partido entre equipo local " + local.getNombre()
            + " y equipo visitante " + visitante.getNombre()
            + " todavía no se ha jugado";
    }
    return "Partido entre equipo local " + local.getNombre()
        + " y el equipo visitante " + visitante.getNombre()
        + " jugado en el estadio " + local.getEstadio()
        + " de la ciudad " + local.getCiudad()
        + " ha finalizado con " + golesLocal + " goles de equipo local"
        + " y " + golesVisitante + " goles de equipo visitante."
        + " Resultado quiniela= " + quiniela;
}

}

public class MainLiga {
    public static void main(String[] args) {
        Equipo rm = new Equipo("Real Madrid", "Bernabéu", "Madrid");
        Equipo fcb = new Equipo("Barcelona", "Camp Nou", "Barcelona");
        Equipo atm = new Equipo("Atlético", "Wanda", "Madrid");

        // Partido 1
        Partido p1 = new Partido(1, rm, fcb);
        try {
            p1.ponerResultado("2-1");
        } catch (Exception e) {
            System.out.println("Error p1: " + e.getMessage());
        }

        // Partidos 2 y 3 (reintento si hay error)
        Partido p2 = null, p3 = null;
        boolean ok = false;
        while (!ok) {
            try {
                p2 = new Partido(2, fcb, atm);
                p3 = new Partido(3, atm, rm);
                p2.ponerResultado("0-0");
                p3.ponerResultado("1-2");
                ok = true;
            } catch (Exception e) {
                System.out.println("Error creando partidos: " + e.getMessage());
            }
        }

        System.out.println(p1);
        System.out.println(p2);
        System.out.println(p3);
        System.out.println(rm);
    }
}

```

```
        System.out.println(fcb);  
        System.out.println(atm);  
    }  
}
```